



# HDI Predict: Machine Learning-Based Human Development Index Classifier

Documentation format adapted from the SmartBridge & SkillWallet *Sample Project Documentation* template.

## Project Description:

HDI Predict harnesses supervised machine learning to estimate a country's Human Development Index (HDI) classification directly from its core development indicators. The platform includes a Prediction Engine, which classifies a country into one of four development tiers — Very High, High, Medium, or Low — a Feature Insight module, which reports how life expectancy, education, and income indicators individually influenced the prediction, and a Comparison Dashboard, which benchmarks the entered country profile against historical regional averages.

The model is trained on a curated dataset built from publicly available UNDP Human Development Report indicators: life expectancy at birth, mean years of schooling, expected years of schooling, and Gross National Income (GNI) per capita (PPP). A Random Forest Classifier, selected for its strong performance on mixed-scale tabular data and its resistance to overfitting, is trained using scikit-learn and persisted with joblib. The trained model is served through a Flask backend and exposed publicly via Ngrok, giving policymakers, researchers, and students an accessible way to explore how development indicators translate into human development outcomes.

## Scenarios

**Scenario 1: Predicting Very High Human Development.** A user selects a country with high life expectancy, strong mean years of schooling, expected years of schooling, and a high GNI per capita. Based on these indicators, the model predicts a **Very High HDI** score, classifying the country among the most developed nations. This helps users understand how strong performance across multiple development dimensions contributes to overall human development.

**Scenario 2: Identifying Development Gaps in Emerging Economies.** A policymaker inputs mid-range values representing a developing nation, including moderate life expectancy, average educational attainment, and a moderate income level. The model returns a **Medium HDI** score, providing insights into areas where improvements in healthcare, education, or income generation could significantly enhance human development outcomes.

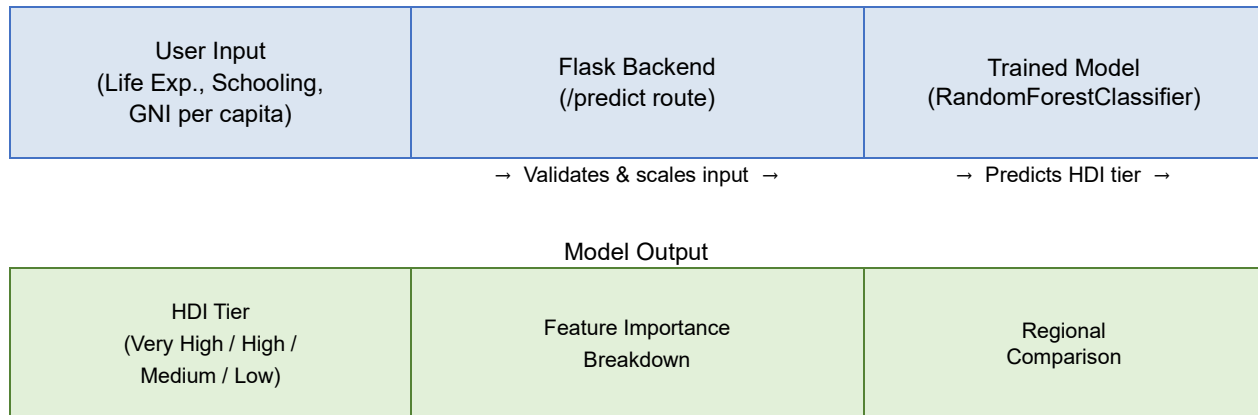
**Scenario 3: Assessing Countries Requiring Development Intervention.** A researcher evaluates a country with relatively low life expectancy, limited educational opportunities, and low GNI per capita. The



model predicts a **Low HDI** score and highlights the country's developmental challenges. Such predictions can support governments and development organizations in prioritizing investments and policy interventions.

## Technical Architecture:

The architecture below shows how a user's indicator values flow through the system to produce a classification and an explanation.



**Description:** HDI Predict uses a modular three-tier architecture to deliver rapid, ML-driven classification. The frontend provides a responsive form built with HTML and CSS for entering the four core indicators. The Flask-based backend validates and scales the input, then passes it to a pre-trained scikit-learn pipeline loaded at startup via joblib. The pipeline returns both the predicted HDI tier and per-feature contribution scores, which the backend packages into a JSON response for the frontend to render, with deployment managed locally via Flask and exposed publicly through Ngrok.

**Note:** Since this project persists a trained model file (hdi\_model.pkl) and a reference dataset rather than a relational database, no ER Diagram is required. If a future version introduces user accounts or a prediction-history database, an ER Diagram should be added alongside its description.

## Pre-requisites:

- **Python Programming Proficiency:** Python Documentation
- **Machine Learning Fundamentals:** scikit-learn User Guide
- **Pandas & NumPy for Data Handling:** Pandas Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide
- **Ngrok for Public Deployment:** Ngrok Documentation

## Project Workflow:

### Milestone 1: Data Preparation and Model Selection

- **Activity 1.1:** Collect and clean UNDP HDI indicator data (life expectancy, mean years of schooling, expected years of schooling, GNI per capita) into a structured CSV dataset.

- **Activity 1.2:** Research and select the appropriate classification algorithm for predicting HDI tiers from the four indicators.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and the trained model.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, scikit-learn, and pandas.

## Milestone 2: Model Training and Core Functionalities

- **Activity 2.1:** Preprocess the dataset (scaling, train/test split) and train the RandomForestClassifier to predict HDI tier.
- **Activity 2.2:** Evaluate the model using accuracy, precision, recall, and a confusion matrix, then persist the trained pipeline with joblib.

## Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for the home page and the /predict endpoint, and loading the persisted model at startup.

## Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic result rendering with JavaScript to display the predicted tier, confidence score, and feature importance chart.

## Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

## Milestone 6: Conclusion

### Milestone 1: Data Preparation and Model Selection

In this milestone, the focus is on assembling a reliable indicator dataset and selecting the classification approach that best predicts HDI tier from a country's development indicators.

#### Activity 1.1: Collect and Clean the HDI Dataset

- Source historical indicator values (life expectancy, mean years of schooling, expected years of schooling, GNI per capita PPP) for a broad set of countries and years.

- Label each row with its official HDI tier — Very High, High, Medium, or Low — based on published UNDP thresholds.
- Clean missing values, standardize units, and export the result as `hdi_dataset.csv`.

## Activity 1.2: Research and Select the Appropriate Model

- **Understand the Project Requirements:** Review the specific needs of the HDI Predict application, focusing on multi-class classification across four ordered tiers.
- **Explore scikit-learn's Model Options:** Examine candidate classifiers, including Logistic Regression, Support Vector Machines, and ensemble tree methods.
- **Evaluate Model Performance:** Compare candidates on validation accuracy, interpretability, and robustness to correlated features (education and income are often correlated).
- **Select the Optimal Model:** Choose a **RandomForestClassifier** (`n_estimators=200`, `max_depth=8`, `random_state=42`) for its strong accuracy, resistance to overfitting, and built-in feature-importance scoring.

## Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend (HTML, CSS, JavaScript), Flask backend, and the trained model pipeline.
- **Detail Frontend Functionality:** Outline how users interact with the application — entering life expectancy, schooling years, and GNI per capita, then submitting for prediction.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to `/predict`, validates numeric ranges, scales input, and queries the model.
- **Describe Model Integration Points:** Define how the backend loads `hdi_model.pkl` and `hdi_scaler.pkl` at startup, transforms input through `predict()` and `predict_proba()`, and returns structured results to the frontend.

## Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.9+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using `venv` to isolate project dependencies.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask, scikit-learn, pandas, and joblib:**

```
(myenv) D:\projects> pip install Flask==3.0.3
(myenv) D:\projects> pip install scikit-learn==1.5.1
(myenv) D:\projects> pip install pandas==2.2.2
```



```
(myenv) D:\projects> pip install joblib==1.4.2
```

- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), a model artifact folder, and main application files (app.py, train\_model.py, requirements.txt, run\_public.py).

## Milestone 2: Model Training and Core Functionalities

### Activity 2.1: Train and Evaluate the Classification Model

Implement `train_model.py` to load the cleaned dataset, scale the four indicators, split the data, train the `RandomForestClassifier`, and persist both the model and the scaler for use by the Flask app.

```
import pandas as pd from sklearn.model_selection import
train_test_split from sklearn.preprocessing import
StandardScaler from sklearn.ensemble import
RandomForestClassifier from sklearn.metrics import
accuracy_score, classification_report import joblib

df = pd.read_csv("data/hdi_dataset.csv") FEATURES = ["life_expectancy",
"mean_schooling", "expected_schooling", "gni_per_capita"] X = df[FEATURES] y =
df["hdi_tier"] # Very High / High / Medium / Low

scaler = StandardScaler()
X_scaled =
scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42,
    stratify=y )

model = RandomForestClassifier(n_estimators=200, max_depth=8, random_state=42)
model.fit(X_train, y_train)

preds = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test,
preds)) print(classification_report(y_test,
preds))

joblib.dump(model, "model/hdi_model.pkl")
joblib.dump(scaler, "model/hdi_scaler.pkl")
```

- **Feature Set:** `life_expectancy`, `mean_schooling`, `expected_schooling`, `gni_per_capita`.
- **Target:** `hdi_tier`, a four-class label (Very High, High, Medium, Low).
- **Persisted Artifacts:** `hdi_model.pkl` (trained classifier) and `hdi_scaler.pkl` (fitted `StandardScaler`), both loaded once when the Flask app starts. **Activity 2.2: Implement the Flask Backend**

#### Define Routes in Flask:

- Set up routes in `app.py` for the home page and the prediction endpoint.

#### Process User Input:

- Create a form in `index.html` for users to submit life expectancy, mean years of schooling, expected years of schooling, and GNI per capita.
- Use Flask's `request.get_json()` to retrieve JSON data submitted from the frontend via AJAX.
- Validate that all four fields are present, numeric, and within realistic bounds (e.g., life expectancy between 30 and 90 years) before processing.

***Integrate Model Calls:***

- Within the /predict route handler, call `build_feature_vector()` to assemble the four indicators in the correct order.
- Transform the vector using the persisted `hdi_scaler.pkl`, then call `model.predict()` and `model.predict_proba()`.
- Pair the prediction with `model.feature_importances_` to report which indicators most influenced the result.

## Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the HDI Predict application. In this milestone, you'll set up the prediction route, load the persisted model and scaler, and return structured, explainable results to the frontend.

### Activity 3.1: Writing the Main Application Logic in app.py

#### *Define the Core Routes in app.py:*

- / — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

- /predict — POST endpoint that accepts JSON input, runs the model, and returns structured results

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()
    try:
        life_expectancy = float(data.get("life_expectancy"))
        mean_schooling = float(data.get("mean_schooling"))
        expected_schooling = float(data.get("expected_schooling"))
        gni_per_capita = float(data.get("gni_per_capita"))
    except (TypeError, ValueError):
        return jsonify({"error": "All four indicators must be valid numbers."}), 400

    if not (20 <= life_expectancy <= 95):
        return jsonify({"error": "Life expectancy must be between 20 and 95 years."}), 400
    if gni_per_capita <= 0:
        return jsonify({"error": "GNI per capita must be a positive value."}), 400

    try:
        features = build_feature_vector(
            life_expectancy,
            mean_schooling, expected_schooling, gni_per_capita
        )
        scaled = scaler.transform([features])

        tier = model.predict(scaled)[0]
        probabilities = model.predict_proba(scaled)[0]
        confidence = round(float(max(probabilities)) * 100, 2)

        importances = dict(zip(FEATURES, model.feature_importances_.tolist()))

        return jsonify({
            "success": True,
            "hdi_tier": tier,
            "confidence": confidence,
            "feature_importance": importances
        })
    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({"error": f"Prediction failed: {str(e)}"}), 500
```

## Define Prediction Helpers:

- **FEATURES** list: fixes the exact order of indicators the scaler and model expect — life\_expectancy, mean\_schooling, expected\_schooling, gni\_per\_capita.
- **build\_feature\_vector()**: assembles the four validated inputs into a single ordered list ready for scaling.

```
FEATURES = ["life_expectancy", "mean_schooling", "expected_schooling", "gni_per_capita"]

def build_feature_vector(life_expectancy, mean_schooling, expected_schooling,
                        gni_per_capita):
    return [life_expectancy, mean_schooling, expected_schooling,
            gni_per_capita]
```

## Load the Model at Startup:



- Load hdi\_model.pkl and hdi\_scaler.pkl once when the Flask app starts, rather than on every request, to keep prediction latency low.

```
import joblib

model = joblib.load("model/hdi_model.pkl")    scaler =
joblib.load("model/hdi_scaler.pkl")
```

- **Home route:** / → renders index.html
- **/predict (POST)** → accepts JSON, returns {success, hdi\_tier, confidence, feature\_importance}

## Milestone 4: Frontend Development

Milestone 4 focuses on developing a clear, data-forward interface for HDI Predict. This involves designing a clean, card-based layout with responsive HTML and CSS, and creating dynamic content rendering with vanilla JavaScript to display the predicted tier, confidence score, and feature importance breakdown.

### Activity 4.1: Designing and Developing the User Interface

#### ***Set Up the Base HTML Structure:***

- Develop the main HTML file (index.html) as the single-page interface, including a header with the HDIPredict brand and tagline, an indicator input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for indicator fields and result section headings, and Google Fonts (Inter) for clean typography.

#### ***Design a Responsive Layout Using CSS:***

- Create a CSS stylesheet (static/css/style.css) implementing a light, data-dashboard aesthetic with a distinct accent color per HDI tier (green for Very High, blue for High, amber for Medium, red for Low).
- Use flexbox and grid layouts to organize the results into a two-column structure (predicted tier and confidence on the left, feature importance chart on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

#### ***Create Separate UI Components for Each Output Type:***

- **Tier Badge:** a large, color-coded badge displaying the predicted HDI tier with its confidence percentage.
- **Indicator Summary Card:** echoes back the four values the user entered for quick verification.
- **Feature Importance Bars:** horizontal bar chart rendered inside a card, ranking indicators by their contribution to the prediction.

### Activity 4.2: Creating Dynamic Content Rendering with JavaScript

#### ***Handle Form Submission Asynchronously:***

- Attach a submit event listener to the prediction form that prevents default form submission and sends the four indicator values as a JSON POST request to /predict via the Fetch API.
- During processing, disable the submit button and show an animated CSS loader in place of the button text.

#### ***Render Results Dynamically:***

- Implement the renderResults(data) function in main.js, which dynamically creates and injects the tier badge, indicator summary, and feature importance bars into their respective containers.



- After rendering, automatically smooth-scroll the viewport to the results section using `scrollIntoView()`.

***Restore UI State After Prediction:***

- In the finally block of the async handler, re-enable the submit button and restore the original button text regardless of success or failure.

## Milestone 5: Deployment

In Milestone 5, the focus is on deploying the HDI Predict application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

### Activity 5.1: Preparing the Application for Local Deployment

#### *Set Up a Virtual Environment:*

- Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

#### *Ensure the Model Artifacts Are Present:*

- Confirm model/hdi\_model.pkl and model/hdi\_scaler.pkl exist — run python train\_model.py first if they are missing.

### Activity 5.2: Local Testing and Verification

#### *Start the Flask Development Server:*

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.
- Interact with the predictor using values representative of all four HDI tiers (Very High, High, Medium, Low) to verify predictions, confidence scores, and feature importance are correctly generated, parsed, and rendered.

### Activity 5.3: Public Deployment via Ngrok

Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

#### *Install Ngrok and pyngrok:*

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from [ngrok.com/download](https://ngrok.com/download).
- Sign up at [ngrok.com](https://ngrok.com) and copy your authtoken from the Ngrok dashboard.

## Configure Your Ngrok Authtoken:

- The run\_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run\_public.py with your own Ngrok authtoken before running.

## Run the Public Deployment Script:

- Instead of running app.py directly, launch run\_public.py to start both the Ngrok tunnel and the Flaskserver:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5050 and prints the live public URL in the terminal:

```
===== YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

- Share this URL with anyone who can access your HDI Predict app directly from their browser without any installation.

## Important Notes:

- debug=False** is required in run\_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run\_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run\_public.py to get a new URL when this happens.

## Conclusion:

HDI Predict is a machine learning platform built with scikit-learn and Flask that turns four core development indicators — life expectancy, mean years of schooling, expected years of schooling, and GNI per capita — into an interpretable Human Development Index classification. Using a RandomForestClassifier trained on curated UNDP indicator data, the application instantly classifies a country's profile into Very High, High, Medium, or Low human development, while also reporting which indicators most influenced that outcome. The project, which included dataset preparation, model training and evaluation, backend and frontend development, and deployment via Ngrok, illustrates how an interpretable classification model and a structured framework can make development analytics accessible to policymakers, researchers, and students alike.

Looking ahead, HDI Predict offers several opportunities for expansion, such as incorporating additional sub-indicators (e.g., gender inequality or inequality-adjusted HDI), enabling batch predictions from uploaded country datasets, visualizing historical HDI trends over time, and implementing user accounts to



save and revisit past predictions. By continuously refining the training dataset and enhancing the explainability of its predictions, the platform is well-positioned to evolve from a specialized classroom and policy tool into a broader human-development analytics assistant.